

# Voice Orchestrator — Mobile Developer Handoff

Everything you need to integrate the Voice Orchestrator API into the SmartHome mobile app. All examples are curl — port them to your platform's HTTP client of choice.

## 1. Credentials

| Item                               | Value                                                                 |
|------------------------------------|-----------------------------------------------------------------------|
| API base URL                       | <code>https://voiceorchestrator.homeadapt.us/api/v1/voice-auth</code> |
| VAPI assistant id                  | <code>1a2904b1-61cf-49da-a804-199d8d39fb9f</code>                     |
| VAPI public key (browser/SDK only) | <code>0145170b-662c-4db1-983d-b3bf8ae1b4</code>                       |
| Mobile API key — iOS               | <code>sk_ios_48b546d143dae04ec9d2c4396ae9155648e80a5ffbf3377</code>   |
| Mobile API key — Android           | <code>sk_and_a4680b4e40ffe9b5133e8118b7d41cf753fa062688bc05a0</code>  |
| Mobile API key — Web               | <code>sk_web_37ceccf66f52a679b34ce1ba53eca44d7f67f2c281d15bc1</code>  |

Keys are static for v1. Compile via your CI secret store; never commit to git. Tomorrow they become short-lived JWTs — header shape stays `Authorization: Bearer <token>`, only the token source moves.

## 2. Authentication

Every REST request needs:

```
Authorization: Bearer <your-platform-key>
```

Missing or wrong key → `401 { "error": "...", "code": "UNAUTHORIZED" }`.

Examples below use a shell variable:

```
KEY=sk_ios_48b546d143dae04ec9d2c4396ae9155648e80a5ffbf3377
BASE=https://voiceorchestrator.homeadapt.us/api/v1/voice-auth
```

### 2.1 App Startup: getting `user_ref` and `home_id` (STOP hardcoding them)

Until now the examples hardcoded `user_ref: "scott_mobile"` and `home_id: "scott_home"`. Those values are now discoverable per user — the app should fetch and cache them on startup instead.

#### Sign-up (new accounts)

The sign-up screen has three required fields plus an **optional "Home code"** field. HomeAdapt gives a user an 8-character invite code (K7QX-4MRP) for their home; entering it at sign-up skips the manual approval step.

```
curl -s -X POST "$BASE/auth/signup" \ -H "Content-Type: application/json" \ -d '{"email": "new@example.com", "password": "min-8-chars", "full_name": "New User", "invite_code": "K7QX-4MRP"}'
```

**With a valid invite\_code** → 201 with "status": "active" and the full login payload (token, user\_ref, homes, default\_home\_id, ... exactly as /auth/login returns). Treat it as a successful login: store the token and go straight to the home screen. The account is already attached to the invited home.

**Without invite\_code** (or empty string) → 201 {"status": "pending\_approval", "email": "...", "message": "...". Accounts start **pending**: the user cannot log in until HomeAdapt activates the account and attaches their home (admin approval — keeps onboarding under our control). Show the returned message and route back to the login screen.

Invite-code rules for the field: accept any case, with or without the dash, and trim whitespace (the server normalises it). Send the field only when the user typed something.

Errors: 400 INVALID\_INVITE (see below — nothing is created, let the user fix the code or clear it and sign up pending), 409 EMAIL\_EXISTS (offer login instead), 400 VALIDATION, 429 RATE\_LIMITED (5 signups/15 min per IP, or 5 bad invite codes/15 min per IP).

INVALID\_INVITE carries a reason you can map to copy:

| reason        | Show                                                    |
|---------------|---------------------------------------------------------|
| unknown       | "That code isn't valid — check it and try again"        |
| expired       | "That code has expired — ask HomeAdapt for a new one"   |
| exhausted     | "That code has already been used"                       |
| revoked       | "That code was cancelled — ask HomeAdapt for a new one" |
| home_inactive | "The home for that code isn't active"                   |

(The server's error string already says the same thing if you'd rather display it verbatim.)

Logging in before activation (correct password) → 403 PENDING\_APPROVAL — show "your account is awaiting activation."

## Login

```
curl -s -X POST "$BASE/auth/login" \ -H "Content-Type: application/json" \ -d '{"email": "scott@example.com", "password": "*****"}'
```

No Authorization header needed on this call. username is accepted in place of email. Response 200:

```
{ "token": "eyJhbGciOiJIUzI1NiIsInR5cGU": "Bearer", "expires_in": 2592000, "user_ref": "scott_mobile", "user_id": "scott_mobile", "username": "scott", "email": "scott@example.com", "full_name": "Scott", "homes": [ { "home_id": "scott_home", "name": "Scott's House" } ], "default_home_id": "scott_home" }
```

Wrong credentials → 401 {"error": "Invalid credentials", "code": "UNAUTHORIZED"}. After 5 failed attempts the account/IP is locked for 15 minutes → 429 {"error": "Too many failed attempts. Try again later.", "code": "RATE\_LIMITED"} — show a "try again later" message, don't retry automatically.

**Cache** token, user\_ref, and default\_home\_id. Use user\_ref and default\_home\_id everywhere the examples below say scott\_mobile / scott\_home. user\_id and email are included so the app can attach the user's identity to feedback reports.

## Startup flow

1. No stored token → show login screen → POST /auth/login → store token.
2. Have a token → GET /me to refresh identity:

```
curl -s "$BASE/me" -H "Authorization: Bearer $TOKEN"
```

Returns the same identity payload (minus the token fields). 401 means the token expired → re-login.

## Change password

```
curl -s -X POST "$BASE/auth/change-password" \ -H "Authorization: Bearer $TOKEN" \ -H "Content-Type: application/json" \ -d '{"current_password": "old-one", "new_password": "new-one-min-8-chars"}'
```

204 on success (no body). The current token stays valid until it expires — no forced re-login. Errors: 403 FORBIDDEN wrong current password, 400 VALIDATION new password outside 8–256 chars, 429 RATE\_LIMITED after 5 wrong current-password attempts (15-min lock), 401 missing/expired token. Forgotten passwords are reset by the admin (PUT /admin/users/{id}/password).

## Join a home with an invite code (existing accounts)

An already logged-in user can add a home to their account by entering an invite code — e.g. a second tester joining a shared demo home, or a family member being added to a house. Put this behind a **"Join a home"** action (settings or the home picker).

```
curl -s -X POST "$BASE/auth/redeem-invite" \ -H "Authorization: Bearer $TOKEN" \ -H "Content-Type: application/json" \ -d '{"invite_code": "K7QX-4MRP"}'
```

200 → the same identity payload as GET /me, now including the new home in homes[]. Replace the cached homes list with it; default\_home\_id does not change (it stays the user's first home), so switch selectedHomeId to the new home yourself if that's the better UX.

Errors: 400 INVALID\_INVITE (+ reason, same table as sign-up), 400 VALIDATION (missing code), 401 missing/expired token, 429 RATE\_LIMITED after 5 bad codes (15-min lock per account/IP).

## Using the token on every other endpoint

The login token works as the bearer on **all** endpoints below — same header shape as the platform key:

```
Authorization: Bearer <token>
```

Two differences vs the static platform key:

- the server knows who you are: sending a `user_ref` (query or body) that doesn't match the logged-in user, or a `home_id` the user doesn't own, → 403 FORBIDDEN;
- `GET /me` only works with a login token (platform keys carry no identity).

**Transition:** the static platform keys (`sk_ios_...` etc.) continue to work exactly as before — existing builds are unaffected. New app builds should move to the login flow; the static keys will be retired after that.

## 2.2 Multiple homes — selecting & switching

A user can belong to **several homes** (e.g. Scott: house + office), and a home can be **shared by several users** (Scott and Aaron both see `scott_home`). The server has **no "current home"** — the login token identifies the *user*, and every data endpoint takes an explicit `home_id`. Switching is purely client state.

Login / `GET /me` now return all homes:

```
{ "homes": [ { "home_id": "scott_home", "name": "Scott's House" }, { "home_id": "ne_qli_1", "name": "NE QLI Office" } ], "default_home_id": "scott_home" }
```

| Rule                       | Contract                                                                                                                                                                                                         |
|----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Source of truth            | <code>homes[]</code> from login / <code>/me</code> — never hardcode <code>home_id</code>                                                                                                                         |
| Default                    | <code>default_home_id</code> = the user's <b>stored preference</b> if they set one (see <i>Changing the default</i> below), otherwise their <b>first-registered</b> home; stable — adding homes never changes it |
| Selected home              | persist a <code>selectedHomeId</code> , initialized to <code>default_home_id</code> ; switching = change it + refetch (no re-login, token unchanged)                                                             |
| On launch                  | refresh <code>/me</code> ; stored <code>selectedHomeId</code> missing from <code>homes[]</code> → fall back to <code>default_home_id</code> ; <code>homes</code> empty → "awaiting home setup" screen            |
| Picker UI                  | only when <code>homes.length &gt; 1</code> (single-home users see no change); list by name, checkmark the selected                                                                                               |
| Per-home data              | favorites, boards, device search, automations, voice-auth enrollments, VAPI numbers are all keyed by ( <code>user_ref</code> , <code>home_id</code> ) — a freshly added home starts with an empty favorites list |
| Wrong <code>home_id</code> | user is not a member of it → 403 — treat as "re-sync <code>/me</code> ", not an auth failure                                                                                                                     |
| Gaining a home             | admin adds the user, or the user redeems an invite code (POST <code>/auth/redeem-invite</code> ) → <code>homes[]</code> grows on the next <code>/me</code> ; other members of that home are unaffected           |

On switch: cancel in-flight requests, refetch GET /favorites?home\_id=..., GET /dashboards/config?..., and any cached /devices/discover data. A voice-gated favorite in home B needs its own enrollment under home B.

## Changing the default (follows the account, not the device)

selectedHomeId is per device. When the user wants a home to be their starting home everywhere — new phone, reinstall — offer **"Make this my default"** in the home picker and call:

```
curl -s -X PATCH "$BASE/me" \ -H "Authorization: Bearer $TOKEN" \ -H "Content-Type: application/json" \ -d '{"default_home_id": "ne_qli_1"}'
```

200 → the same payload as GET /me with the new default\_home\_id. Apply the response rather than assuming success. Send {"default\_home\_id": null} to go back to "no preference" (first-registered home).

Rules: the home must be one of the user's **active** homes → otherwise 400 VALIDATION. If the user later loses access to that home, the server silently falls back to the first-registered home; the preference returns if they are re-added. 401 without a login token. Admins can also set it from the dashboard (pin icon on the user row) or PUT /admin/users/{id}.

---

## 3. Endpoint Reference

The API has six feature groups:

1. **Voice-auth enrollments** — gate an automation behind a voice phrase challenge
2. **Devices, search & favorites** — discover, search, pin and fire HA items
3. **Scene mappings** — register HA webhook scenes
4. **Automations trigger** — fire a scene/script directly (with voice-gate guard)
5. **Voice enable** — provision a VAPI phone number for a user

Plus the **VAPI SDK** for the spoken challenge UX.

---

### 3.1 Voice-Auth Enrollments

User toggles "Voice Protect" on a scene → enrollment row created. After that, direct triggers for that scene are blocked until the user passes a voice challenge.

#### Create an enrollment

```
curl -s -X POST "$BASE/enrollments" \ -H "Authorization: Bearer $KEY" \ -H "Content-Type: application/json" \ -d '{"user_ref": "scott_mobile", "home_id": "scott_home", "automation_name": "Main Lights On", "ha_service": "script", "ha_entity": "main_lights_on"}'
```

ha\_service ∈ {scene, script, switch, light, lock, cover, media\_player, climate, input\_boolean, fan}. ha\_entity is the **suffix only** — main\_lights\_on, **not** script.main\_lights\_on.

Response 201:

```
{ "id": "a8100373-7f6c-4e27-9ecf-71d19c1ed4cd", "user_ref": "scott_mobile", "home_id": "scott_home", "automation_id": "main_lights_on", "automation_name": "Main Lights On", "ha_service": "script", "ha_entity": "main_lights_on", "status": "ACTIVE", "max_attempts": 3, "cooldown_seconds": 30 }
```

Idempotent on `(user_ref, automation_id)`.

## List enrollments

```
curl -s -H "Authorization: Bearer $KEY" \ "$BASE/enrollments?user_ref=scott_mobile"
```

Response 200:

```
{ "count": 1, "items": [ { "id": "a8100373-...", "automation_name": "Main Lights On", "status": "ACTIVE" } ] }
```

## Pre-flight check before launching VAPI session

```
curl -s -H "Authorization: Bearer $KEY" \ "$BASE/check?user_ref=scott_mobile&automation_id=main_lights_on"
```

Response 200:

```
{ "exists": true, "enrollment_id": "a8100373-...", "status": "ACTIVE", "cooldown_remaining_seconds": 0, "attempts_remaining": 3, "enrollment_required": false }
```

If `exists: false` → user hasn't enrolled this automation; show enrollment UI. If `cooldown_remaining_seconds > 0` → display countdown, block voice session.

## Pause / resume / revoke

```
# Pause curl -s -X PATCH "$BASE/enrollments/<id>/status" \ -H "Authorization: Bearer $KEY" -H "Content-Type: application/json" \ -d '{"status":"PAUSED"}' # Resume curl -s -X PATCH "$BASE/enrollments/<id>/status" \ -H "Authorization: Bearer $KEY" -H "Content-Type: application/json" \ -d '{"status":"ACTIVE"}' # Permanently revoke (returns 204) curl -s -X DELETE "$BASE/enrollments/<id>" -H "Authorization: Bearer $KEY"
```

## Recent challenge attempts (audit log)

```
curl -s -H "Authorization: Bearer $KEY" \ "$BASE/challenges?user_ref=scott_mobile&limit=10"
```

Each entry records `result` (PASS / FAIL / TIMEOUT) and `vapi_call_id` for correlating with VAPI dashboards.

---

## 3.2 Devices, Search and Favorites

Per-user pinned HA items — devices (Den Lamp, Yale Lock), scenes (Good Morning), scripts, and HA-configured automations. Drives the home-screen tile grid.

### 3.2.1 Discover physical devices

```
curl -s -H "Authorization: Bearer $KEY" \ "$BASE/devices/discover?home_id=scott_home"
```

Returns one row per HA *device* (not per entity). Each row includes `device_id`, `name`, `manufacturer`, `model`, `area`, the resolved `primary_entity_id` (the controllable surface — light/switch/lock/...) and the full list of entities bound to that device.

Response 200:

```
{ "home_id": "scott_home", "count": 65, "items": [ { "device_id":  
"ca96f20dd98e20c6d43a791775940295", "name": "Yale Lock", "manufacturer": "Yale", "model":  
"YRD226 TSDB", "area": "Entry", "primary_entity_id": "lock.yale_yrd226_tsdb", "primary_domain":  
"lock", "is_controllable": true, "all_entities": ["lock.yale_yrd226_tsdb"] } ] }
```

Cached server-side for 60s per home. Devices with only sensor/diagnostic entities have `is_controllable`: `false` and cannot be favorited.

### 3.2.2 Unified search — devices, scenes, scripts, automations

The single endpoint behind the mobile "Add favorite" picker.

```
curl -s -G "$BASE/items/search" \ -H "Authorization: Bearer $KEY" \ --data-urlencode  
"home_id=scott_home" \ --data-urlencode "q=bat" \ --data-urlencode "user_ref=scott_mobile"
```

Query params:

| Param                 | Required | Notes                                                                            |
|-----------------------|----------|----------------------------------------------------------------------------------|
| <code>home_id</code>  | yes      | which home to search                                                             |
| <code>q</code>        | no       | case-insensitive substring on name + entity_id + device_id                       |
| <code>kind</code>     | no       | comma-separated filter from <code>device,scene,script,automation,entity</code>   |
| <code>user_ref</code> | no       | when set, populates <code>is_favorited</code> + <code>favorite_id</code> per row |
| <code>limit</code>    | no       | default 200, max 500                                                             |

Response 200:

```
{ "home_id": "scott_home", "count": 4, "items": [ { "kind": "device", "device_id":  
"6b86cd8c539ad69b193a8ff2acbf3b4e", "entity_id": "switch.bat_sign", "name": "Bat Sign",  
"domain": "switch", "manufacturer": "Belkin", "model": "Socket", "area": "Downstairs Bat Cave",  
"state": "off", "is_favorited": true, "favorite_id": "26260ec6-ccfb-4cfb-a84c-55a3b458dc23" }, {  
"kind": "automation", "device_id": null, "entity_id": "automation.utogglebat_sign", "name":  
"uBatSignToggle", "domain": "automation", "state": "on", "is_favorited": false, "favorite_id":  
null } ] }
```

When `is_favorited: true`, `favorite_id` gives you the UUID needed to toggle off via `DELETE /favorites/{id}` — no second roundtrip needed.

**One-step delete by device/entity:** `DELETE /favorites/{ref}` also accepts a `device_id` or `entity_id` as the ref — add `?user_ref=...&home_id=...` so the server knows whose favorite to remove (with a login token, `user_ref` is inferred; only `home_id` is needed):

```
curl -s -X DELETE -H "Authorization: Bearer $KEY" \
"$BASE/favorites/6b86cd8c539ad69b193a8ff2acbf3b4e?user_ref=scott_mobile&home_id=scott_home"
```

204 on success; 404 if that user has no favorite matching the ref.

### 3.2.3 Add a favorite — by device OR by entity

**Required fields:** `user_ref`, `home_id`, and **exactly one** of `device_id` or `entity_id`. Everything else is optional.

| Field                      | When                           | Notes                                                                                                                                                                   |
|----------------------------|--------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>user_ref</code>      | always                         | scopes the favorite to one user                                                                                                                                         |
| <code>home_id</code>       | always                         | scopes the favorite to one home                                                                                                                                         |
| <code>device_id</code>     | for physical things            | server resolves entity + name from HA registry                                                                                                                          |
| <code>entity_id</code>     | for scenes/scripts/automations | format <code>domain.suffix</code>                                                                                                                                       |
| <code>friendly_name</code> | optional                       | for <code>device_id</code> adds: defaults to HA's device name. for <code>entity_id</code> adds: defaults to the entity suffix (slug) — usually worth setting explicitly |
| <code>position</code>      | optional                       | defaults to next-after-last                                                                                                                                             |

**Favoriting a device** (no `friendly_name` needed — server pulls it from HA):

```
curl -s -X POST "$BASE/favorites" -H "Authorization: Bearer $KEY" -H "Content-Type: application/json" -d '{ "user_ref": "scott_mobile", "home_id": "scott_home", "device_id": "6b86cd8c539ad69b193a8ff2acbf3b4e" }'
```

The server resolves `entity_id`, `primary_entity_id`, `domain`, and `friendly_name` ("Bat Sign") from the HA device registry. You may still override `friendly_name` if the user has renamed the device in your app.

**Favoriting a scene / script / HA automation** (no device behind them — `friendly_name` recommended so you don't get the slug):

```
curl -s -X POST "$BASE/favorites" \ -H "Authorization: Bearer $KEY" -H "Content-Type: application/json" \ -d '{ "user_ref": "scott_mobile", "home_id": "scott_home", "entity_id": "scene.good_morning", "friendly_name": "Good Morning" }'
```

kind is inferred from the domain prefix (scene/script/automation/entity).

**Important constraints:** - Send **exactly one** of `device_id` or `entity_id` — both → 400, neither → 400. - Devices with only sensors/diagnostics → 400 `NO_CONTROLLABLE_ENTITY`. - Duplicate (`user_ref`, `home_id`, `entity_id`) → 400. -

**Locks auto-enroll:** when the resolved entity is `lock.*`, the server creates a STEP\_UP voice-auth enrollment in the same transaction. The response includes `voice_auth_required: true` and `voice_auth_enrollment_id`.

Response 201 (lock device example):

```
{ "id": "440c4234-4fea-4c1c-8717-425e7afdfdea", "user_ref": "scott_mobile", "home_id": "scott_home", "entity_id": "lock.yale_yrd226_tsdb", "friendly_name": "Yale Lock", "domain": "lock", "kind": "device", "device_id": "ca96f20dd98e20c6d43a791775940295", "primary_entity_id": "lock.yale_yrd226_tsdb", "position": 3, "voice_auth_required": true, "voice_auth_enrollment_id": "9c520ce7-bdb5-4143-8e59-091b0626bdf5", "created_at": "2026-04-30T20:31:46.989419" }
```

### 3.2.4 List favorites

```
curl -s -H "Authorization: Bearer $KEY" \
"$BASE/favorites?user_ref=scott_mobile&home_id=scott_home"
```

Each row includes `voice_auth_required` (computed at read time from the enrollments table) so the mobile UI can render the lock badge correctly.

Response:

```
{ "count": 4, "items": [ { "id": "...", "kind": "entity", "entity_id": "switch.bat_sign", "voice_auth_required": false }, { "id": "...", "kind": "scene", "entity_id": "scene.good_morning", "voice_auth_required": false }, { "id": "...", "kind": "automation", "entity_id": "automation.lights_off_at_night", "voice_auth_required": false }, { "id": "...", "kind": "device", "entity_id": "lock.yale_yrd226_tsdb", "voice_auth_required": true } ] }
```

Items are ordered by `position`.

### 3.2.5 Fire a favorite (one-tap activation)

```
curl -s -X POST "$BASE/favorites/<id>/fire" \ -H "Authorization: Bearer $KEY" -H "Content-Type: application/json" \ -d '{}'
```

The server resolves the HA action automatically per domain: - scenes / scripts / lights / switches → `turn_on` - HA automations → `trigger` - locks → `unlock`

Response 200 on success:

```
{ "success": true, "message": "ok", "status_code": 200, "latency_ms": 142, "favorite_id": "...", "entity_id": "switch.bat_sign" }
```

**Voice-gate guard — 409 ENROLLMENT\_REQUIRED:**

```
{ "error": "this favorite requires voice authentication", "code": "ENROLLMENT_REQUIRED", "enrollment_id": "9c520ce7-...", "automation_name": "Yale Lock", "automation_id": "yale_yrd226_tsdb", "home_id": "scott_home" }
```

When you see 409, route the user into the VAPI session (§3.6) using the returned `automation_id` and `home_id` — the lock will be unlocked only after the spoken phrase challenge passes. **This is non-bypassable for locks** — every lock favorite is auto-enrolled.

### 3.2.6 Remove a favorite

```
curl -s -X DELETE "$BASE/favorites/<id>" -H "Authorization: Bearer $KEY"
```

Returns 204 on success, 404 if id is unknown.

### 3.2.7 Reorder (drag-and-drop UI)

```
curl -s -X PATCH "$BASE/favorites/reorder" \ -H "Authorization: Bearer $KEY" -H "Content-Type: application/json" \ -d '{ "items": [ {"id": "<id-1>", "position": 0}, {"id": "<id-2>", "position": 1} ] }'
```

### 3.2.8 Worked example — search, add, remove a device

End-to-end round-trip verified on production. Copy-paste, replace `USER` with your test user, and run top-to-bottom:

```
KEY=sk_ios_48b546d143dae04ec9d2c4396ae9155648e80a5ffbf3377  
BASE=https://voiceorchestrator.homeadapt.us/api/v1/voice-auth USER=demo_user_42 HOME=scott_home
```

**Step 1 — search for devices matching "bat".** Returns each with its `device_id`, name, manufacturer, current state, and per-user `is_favorited` flag.

```
curl -s -G "$BASE/items/search" \ -H "Authorization: Bearer $KEY" \ --data-urlencode  
"home_id=$HOME" \ --data-urlencode "kind=device" \ --data-urlencode "q=bat" \ --data-urlencode  
"user_ref=$USER"
```

Real response from prod (3 devices):

```
name device_id state Bat Cave Echo Show 5 d97e38a608fe463cd3cf0ea4a8e38850 unavailable Bat Sign  
6b86cd8c539ad69b193a8ff2acbf3b4e off Bathroom 71525ebcb48461b3bc93d2300994afe6 playing
```

**Step 2 — add Bat Sign to favorites by device\_id.** No need to send `entity_id` or `friendly_name` — the server resolves both from the HA registry.

```
curl -s -X POST "$BASE/favorites" -H "Authorization: Bearer $KEY" -H "Content-Type: application/json" -d  
"{\"user_ref\": \"$USER\", \"home_id\": \"$HOME\", \"device_id\": \"6b86cd8c539ad69b193a8ff2acbf3b4e\"}"
```

Response 201:

```
{ "id": "808dc489-eced-4327-bc60-ab1a690997b3", "user_ref": "demo_user_42", "home_id":  
"scott_home", "entity_id": "switch.bat_sign", "friendly_name": "Bat Sign", "domain": "switch",  
"kind": "device", "device_id": "6b86cd8c539ad69b193a8ff2acbf3b4e", "primary_entity_id":  
"switch.bat_sign", "position": 0, "voice_auth_required": false, "created_at":  
"2026-04-30T21:56:59.422121" }
```

Save the returned `id` (808dc489-...) — it's the favorite UUID you'll use for delete and fire.

**Step 3 — remove the favorite by its UUID.**

```
curl -s -X DELETE "$BASE/favorites/808dc489-eced-4327-bc60-ab1a690997b3" \ -H "Authorization: Bearer $KEY" -w "HTTP %{http_code}\n"
```

Response: HTTP 204 (empty body). A subsequent GET `/favorites?user_ref=$USER&home_id=$HOME` will show `count: 0`.

#### Notes for the implementation:

- For a *lock* device (e.g. `device_id` of the Yale Lock), step 2 returns `voice_auth_required: true` and includes `voice_auth_enrollment_id`. Tapping the tile next must hit `/favorites/{id}/fire`, which returns 409 `ENROLLMENT_REQUIRED` — route to VAPI from there.
- For a *scene/script/automation*, send `entity_id` instead of `device_id` in step 2: `{"user_ref": ..., "home_id": ..., "entity_id": "scene.good_morning"}`.
- The mobile app should always re-fetch favorites via GET `/favorites?user_ref=...&home_id=...` after any add/remove — the server is the single source of truth (no client-side cache).

---

## 3.3 Scene Mappings

Map a friendly scene name (e.g. `"decorations on"`) to an HA webhook id. This is what lets Alexa or VAPI translate spoken names into webhook calls.

### Create a scene mapping

```
curl -s -X POST "$BASE/scene-mappings" \ -H "Authorization: Bearer $KEY" -H "Content-Type: application/json" \ -d '{"home_id": "scott_home", "scene_name": "Movie Night", "webhook_id": "movie_night_1751404299018" }'
```

Scene names are normalized to lowercase server-side.

Response 201:

```
{ "id": "3e849ff7-be41-4bd2-aae4-eb139f3a0c9b", "home_id": "scott_home", "scene_name": "movie night", "webhook_id": "movie_night_1751404299018", "is_active": true, "created_at": "2026-04-28T15:30:09.818419" }
```

### List scenes for a home

```
curl -s -H "Authorization: Bearer $KEY" \ "$BASE/scene-mappings?home_id=scott_home"
```

### Update a scene (rename, swap webhook, or deactivate)

```
curl -s -X PATCH "$BASE/scene-mappings/<id>" \ -H "Authorization: Bearer $KEY" -H "Content-Type: application/json" \ -d '{"webhook_id": "new_webhook_id_here"}
```

Body fields are all optional: `scene_name`, `webhook_id`, `is_active`.

### Delete

```
curl -s -X DELETE "$BASE/scene-mappings/<id>" -H "Authorization: Bearer $KEY"
```

---

---

## 3.4 Automation Trigger

Direct fire-and-forget activation. Used when the user taps a favorite tile or a "scenes" button. **Refuses with 409 if a voice-auth enrollment gates the automation** — the app must then route through the VAPI session instead.

### Fire an automation

```
curl -s -X POST "$BASE/automations/trigger" \ -H "Authorization: Bearer $KEY" -H "Content-Type: application/json" \ -d '{ "home_id": "scott_home", "ha_service": "scene", "ha_entity": "movie_night", "user_ref": "scott_mobile", "automation_id": "movie_night" }'
```

`user_ref` and `automation_id` are **required** for the voice-gate check — omit them and the gate is bypassed (only do that for clearly non-protected items).

Response 200:

```
{ "success": true, "message": "OK", "status_code": 200, "latency_ms": 142 }
```

Response 409 (voice-auth enrollment exists for this automation):

```
{ "error": "this automation requires voice authentication", "code": "ENROLLMENT_REQUIRED", "enrollment_id": "a8100373-..." }
```

When you see 409 `ENROLLMENT_REQUIRED`, start the VAPI SDK with the appropriate `variableValues` (see §3.6) instead of retrying this endpoint.

Response 502 if Home Assistant is unreachable; 400 for validation errors.

---

## 3.5 Voice Enable (VAPI provisioning)

Provision a dedicated phone number for the user via VAPI. After this, the user can place outbound calls TO that number to authenticate by voice.

### Enable voice for a user

```
curl -s -X POST "$BASE/voice-enable" \ -H "Authorization: Bearer $KEY" -H "Content-Type: application/json" \ -d '{ "user_ref": "scott_mobile", "home_id": "scott_home", "area_code": "415" }'
```

`area_code` is optional; if omitted, VAPI picks one.

Response 201:

```
{ "id": "432da968-9f19-4fba-b92c-36bee112c7f1", "user_ref": "scott_mobile", "home_id": "scott_home", "phone_e164": "+14151702028", "vapi_phone_number_id": "vpn_abc123...", "is_active": true }
```

**Idempotent** — calling again for the same (`user_ref`, `home_id`) returns the existing mapping without a second VAPI purchase. **Billable on the VAPI side** when first called.

## Status check

```
curl -s -H "Authorization: Bearer $KEY" \ "$BASE/voice-enable?user_ref=scott_mobile"
```

Response 200:

```
{ "enabled": true, "is_dry_run": false, "mapping": { "phone_e164": "+14151702028",  
  "vapi_phone_number_id": "vpn_abc123..." } }
```

`is_dry_run: true` means the server hasn't been configured with a live VAPI key — useful for staging environments. Production should always show `false`.

## Disable (release the number)

```
curl -s -X DELETE "$BASE/voice-enable?user_ref=scott_mobile" \ -H "Authorization: Bearer $KEY"
```

Returns 204 on success, 404 if no active mapping.

---

## 3.6 VAPI SDK — Voice Challenge

When the user taps a voice-protected automation (or after a 409 `ENROLLMENT_REQUIRED` from `/automations/trigger`), launch the VAPI SDK with the public key and these `variableValues`:

```
assistantId: "1a2904b1-61cf-49da-a804-199d8d39fb9f" assistantOverrides.variableValues: home_id:  
"scott_home" user_ref: "scott_mobile" automation_id: "main_lights_on" automation_name: "Main  
Lights On" initiated_by: "MOBILE_IOS" | "MOBILE_ANDROID"
```

The assistant says *"Confirm Main Lights On? Say the security phrase."* — the user repeats — Home Assistant fires. Subscribe to VAPI's `call-end` event to know when it's done; then refresh enrollment state with `GET /check`.

VAPI SDK references: - iOS: <https://github.com/VapiAI/ios> - Android: <https://github.com/VapiAI/android> - Web: <https://github.com/VapiAI/web>

The SDK uses the **VAPI public key**, not the mobile API key.

---

## 4. Error Reference

| HTTP            | Code           | Meaning                                                                  | App action                                          |
|-----------------|----------------|--------------------------------------------------------------------------|-----------------------------------------------------|
| 200 / 201 / 204 | —              | OK                                                                       | proceed                                             |
| 400             | VALIDATION     | Bad request body                                                         | fix input, show inline error                        |
| 400             | INVALID_INVITE | Invite code unknown /<br>expired / exhausted /<br>revoked (reason field) | show reason, let user<br>re-enter or clear the code |

|     |                        |                                                                    |                                                                             |
|-----|------------------------|--------------------------------------------------------------------|-----------------------------------------------------------------------------|
| 401 | UNAUTHORIZED           | Missing / wrong API key or login credentials                       | auth-config bug — do not retry blindly; on login show "invalid credentials" |
| 403 | PENDING_APPROVAL       | Correct password, account not yet activated                        | show "awaiting activation"                                                  |
| 409 | EMAIL_EXISTS           | Sign-up with an email that already has an account                  | offer "log in instead"                                                      |
| 429 | RATE_LIMITED           | Too many failed logins / sign-ups / bad invite codes (15-min lock) | show "try again later"; never auto-retry                                    |
| 404 | —                      | Not found                                                          | varies (prompt to create, etc.)                                             |
| 409 | ENROLLMENT_REQUIRED    | Automation/favorite is voice-gated (always for locks)              | launch VAPI session instead                                                 |
| 400 | NO_CONTROLLABLE_ENTITY | Device has only sensors/diagnostics; cannot be favorited           | exclude from picker                                                         |
| 409 | CONFLICT               | State conflict (e.g. un-revoke)                                    | surface message                                                             |
| 502 | VAPI_ERROR             | VAPI provisioning failed                                           | retry once with backoff                                                     |
| 502 | —                      | Home Assistant unreachable                                         | retry once with backoff                                                     |
| 503 | NOT_CONFIGURED         | Server feature not wired up                                        | report to backend team                                                      |

Error envelope is always:

```
{ "error": "<human message>", "code": "<OPTIONAL_CODE>" }
```

## 5. Acceptance Checklist

Run through these to verify the integration end-to-end:

- `POST /auth/signup` without a code → 201 `pending_approval`; login → 403 `PENDING_APPROVAL`
- `POST /auth/signup` with a valid invite code → 201 `status: active` + token, `homes[]` contains the invited home
- `POST /auth/signup` with a bad invite code → 400 `INVALID_INVITE` and no account created (same email signs up fine afterwards)
- `POST /auth/redeem-invite` with bearer + valid code → 200, `homes[]` grew; `GET /me` agrees
- Two accounts sharing one home both list it at login

- ■ PATCH /me {"default\_home\_id": <other home>} → 200; next login returns that home as default\_home\_id; null reverts to the first-registered home
- ■ GET /enrollments?user\_ref=demo\_user with no Authorization → 401
- ■ Same call with valid bearer → 200 { count: 0, items: [] }
- ■ POST /enrollments → 201 with automation\_id echoed back
- ■ GET /check?user\_ref=...&automation\_id=... → exists: true, attempts\_remaining: 3
- ■ GET /automations/discover?home\_id=... → list of HA candidates
- ■ GET /devices/discover?home\_id=... → list of physical HA devices
- ■ GET /items/search?home\_id=...&q=bat&user\_ref=... → mixed devices/scenes/automations with is\_favorited flag
- ■ POST /favorites { device\_id } → 201, server resolves primary entity
- ■ POST /favorites { entity\_id: "scene.good\_morning" } → 201, kind=scene
- ■ POST /favorites for a sensor-only device → 400 NO\_CONTROLLABLE\_ENTITY
- ■ POST /favorites for a lock.\* → 201 with voice\_auth\_required: true and voice\_auth\_enrollment\_id populated
- ■ POST /favorites/{lock\_fav\_id}/fire → 409 ENROLLMENT\_REQUIRED (never 200)
- ■ GET /favorites → ordered list, lock row has voice\_auth\_required: true
- ■ POST /automations/trigger for an **un-enrolled** automation → 200 success: true
- ■ POST /automations/trigger for an **enrolled** automation → 409 ENROLLMENT\_REQUIRED
- ■ Launch VAPI SDK after a 409 → assistant prompts for security phrase → HA fires
- ■ POST /voice-enable → 201 with phone\_e164; second call same payload → idempotent same id
- ■ DELETE /voice-enable?user\_ref=... → 204; status flips to enabled: false

---

## 6. Contact

When something misbehaves, send the backend team: - The user\_ref you used - A vapi\_call\_id from GET /challenges?user\_ref=<you> if it was a voice flow - The HTTP request + response (curl -i output is ideal)

All tool-call payloads are server-logged with redacted bodies — we can trace any specific call by id.